```c
/*-
 * Copyright (c) 2004, 2020 Oracle and/or its affiliates.  All rights reserved.
 *
 * See the file EXAMPLES-LICENSE for license information.
 */

#include "gettingstarted_common.h"

/* Forward declarations */
int usage(void);
int load_vendors_database(STOCK_DBS, char *);
size_t pack_string(char *, char *, size_t);
int load_inventory_database(STOCK_DBS, char *);

int
usage() {
  fprintf(stderr, "example_database_load [-b <path to data files>]");
  fprintf(stderr, " [-h <database_home_directory>]\n");

  fprintf(stderr, "\tNote: Any path specified must end with your");
  fprintf(stderr, " system's path delimiter (/ or \\)\n");
  return (-1);
}

/*
 * Loads the contents of vendors.txt and inventory.txt into
 * Berkeley DB databases. Also causes the itemname secondary
 * database to be created and loaded.
 */
int
main(int argc, char *argv[]) {
  STOCK_DBS my_stock;
  int ch, ret;
  size_t size;
  char *basename, *inventory_file, *vendor_file;

  /* Initialize the STOCK_DBS struct */
  initialize_stockdbs(&my_stock);

  /* Initialize the base path. */
  basename = "./";

  /* Parse the command line arguments */
  while ((ch = getopt(argc, argv, "b:h:")) != EOF)
    switch (ch) {
      case 'h':
        if (optarg[strlen(optarg) - 1] != '/' &&
            optarg[strlen(optarg) - 1] != '\\')
          return (usage());
        my_stock.db_home_dir = optarg;
        break;
      case 'b':
        if (basename[strlen(basename) - 1] != '/' &&
            basename[strlen(basename) - 1] != '\\')
          return (usage());
        basename = optarg;
        break;
      case '?':
      default:
        return (usage());
    }

```

```c
  /* Identify the files that will hold our databases */
  set_db_filenames(&my_stock);

  /* Find our input files */
  size = strlen(basename) + strlen(INVENTORY_FILE) + 1;
  inventory_file = malloc(size);
  snprintf(inventory_file, size, "%s%s", basename, INVENTORY_FILE);

  size = strlen(basename) + strlen(VENDORS_FILE) + 1;
  vendor_file = malloc(size);
  snprintf(vendor_file, size, "%s%s", basename, VENDORS_FILE);

  /* Open all databases */
  ret = databases_setup(&my_stock, "example_database_load", stderr);
  if (ret) {
    fprintf(stderr, "Error opening databases.\n");
    databases_close(&my_stock);
    return (ret);
  }

  ret = load_vendors_database(my_stock, vendor_file);
  if (ret) {
    fprintf(stderr, "Error loading vendors database.\n");
    databases_close(&my_stock);
    return (ret);
  }
  ret = load_inventory_database(my_stock, inventory_file);
  if (ret) {
    fprintf(stderr, "Error loading inventory database.\n");
    databases_close(&my_stock);
    return (ret);
  }

  /* close our environment and databases */
  databases_close(&my_stock);

  printf("Done loading databases.\n");
  return (ret);
}

/*
 * Loads the contents of the vendors.txt file into
 * a database.
 */
int
load_vendors_database(STOCK_DBS my_stock, char *vendor_file) {
  DBT key, data;
  char buf[MAXLINE];
  FILE *ifp;
  VENDOR my_vendor;

  /* Load the vendors database */
  ifp = fopen(vendor_file, "r");
  if (ifp == NULL) {
    fprintf(stderr, "Error opening file '%s'\n", vendor_file);
    return (-1);
  }

  /* Iterate over the vendor file */
  while (fgets(buf, MAXLINE, ifp) != NULL) {
    /* zero out the structure */
    memset(&my_vendor, 0, sizeof(VENDOR));
```

```c
    /* Zero out the DBTs */
    memset(&key, 0, sizeof(DBT));
    memset(&data, 0, sizeof(DBT));

    /*
     * Scan the line into the structure.
     * Convenient, but not particularly safe.
     * In a real program, there would be a lot more
     * defensive code here.
     */
    sscanf(buf,
           "%20[^#]#%20[^#]#%20[^#]#%3[^#]#%6[^#]#%13[^#]#%20[^#]#%20[^\n]",
           my_vendor.name, my_vendor.street,
           my_vendor.city, my_vendor.state,
           my_vendor.zipcode, my_vendor.phone_number,
           my_vendor.sales_rep, my_vendor.sales_rep_phone);

    /* Now that we have our structure we can load it into the database. */

    /* Set up the database record's key */
    key.data = my_vendor.name;
    key.size = (u_int32_t) strlen(my_vendor.name) + 1;

    /* Set up the database record's data */
    data.data = &my_vendor;
    data.size = sizeof(VENDOR);

    /*
     * Note that given the way we built our struct, there's extra
     * bytes in it. Essentially we're using fixed-width fields with
     * the unused portion of some fields padded with zeros. This
     * is the easiest thing to do, but it does result in a bloated
     * database. Look at load_inventory_data() for an example of how
     * to avoid this.
     */

    /* Put the data into the database */
    my_stock.vendor_dbp->put(my_stock.vendor_dbp, 0, &key, &data, 0);
  } /* end vendors database while loop */

  /* Close the vendor.txt file */
  fclose(ifp);
  return (0);
}

/*
 * Simple little convenience function that takes a buffer, a string,
 * and an offset and copies that string into the buffer at the
 * appropriate location. Used to ensure that all our strings
 * are contained in a single contiguous chunk of memory.
 */
size_t
pack_string(char *buffer, char *string, size_t start_pos) {
  size_t string_size;

  string_size = strlen(string) + 1;
  memcpy(buffer + start_pos, string, string_size);

  return (start_pos + string_size);
}

/*
```

```
 * Loads the contents of the inventory.txt file into
 * a database. Note that because the itemname
 * secondary database is associated to the inventorydb
 * (see env_setup() in gettingstarted_common.c), the
 * itemname index is automatically created when this
 * database is loaded.
 */
int
load_inventory_database(STOCK_DBS my_stock, char *inventory_file) {
  DBT key, data;
  char buf[MAXLINE];
  char databuf[MAXDATABUF];
  size_t bufLen, dataLen;
  FILE *ifp;

  /*
   * Rather than lining everything up nicely in a struct, we're being
   * deliberately a bit sloppy here. This function illustrates how to
   * store mixed data that might be obtained from various locations
   * in your application.
   */
  float price;
  int quantity;
  char category[MAXFIELD], name[MAXFIELD];
  char vendor[MAXFIELD], sku[MAXFIELD];

  /* Load the inventory database */
  ifp = fopen(inventory_file, "r");
  if (ifp == NULL) {
    fprintf(stderr, "Error opening file '%s'\n", inventory_file);
    return (-1);
  }

  /*
   * Read the inventory.txt file line by line, saving each line off to
   * the database as we go.
   */
  while (fgets(buf, MAXLINE, ifp) != NULL) {
    /*
     * Scan the line into the appropriate buffers and variables.
     * Convenient, but not particularly safe. In a real
     * program, there would be a lot more defensive code here.
     */
    sscanf(buf,
           "%20[^#]#%20[^#]#%f#%i#%20[^#]#%20[^\n]",
           name, sku, &price, &quantity, category, vendor);

    /*
     * Now pack it into a single contiguous memory location for
     * storage.
     */
    memset(databuf, 0, MAXDATABUF);
    bufLen = 0;
    dataLen = 0;

    /*
     * We first store the fixed-length elements. This makes our code
     * to retrieve this data from the database a little bit easier.
     */

    /* First discover how long the data element is. */
    dataLen = sizeof(float);
```

```
    /* Then copy it to our buffer */
    memcpy(databuf, &price, dataLen);
    /*
     * Then figure out how much data is actually in our buffer.
     * We repeat this pattern for all the data we want to store.
     */
    bufLen += dataLen;

    /* Rinse, lather, repeat. */
    dataLen = sizeof(int);
    memcpy(databuf + bufLen, &quantity, dataLen);
    bufLen += dataLen;

    bufLen = pack_string(databuf, name, bufLen);
    bufLen = pack_string(databuf, sku, bufLen);
    bufLen = pack_string(databuf, category, bufLen);
    bufLen = pack_string(databuf, vendor, bufLen);

    /*
     * Now actually save the contents of the buffer off
     * to our database.
     */

    /* Zero out the DBTs */
    memset(&key, 0, sizeof(DBT));
    memset(&data, 0, sizeof(DBT));

    /*
     * The key is the item's SKU. This is a unique value, so we need
     * not support duplicates for this database.
     */
    key.data = sku;
    key.size = (u_int32_t) strlen(sku) + 1;

    /* The data is the information that we packed into databuf. */
    data.data = databuf;
    data.size = (u_int32_t) bufLen;

    /* Put the data into the database */
    my_stock.vendor_dbp->put(my_stock.inventory_dbp, 0, &key, &data, 0);
  } /* end vendors database while loop */

  /* Cleanup */
  fclose(ifp);

  return (0);
}
```